

CYBER SECURITY
Case Study Report (Unit 4)

AI-Powered Detection and Automated Response to Malicious PowerShell Attacks Using Machine Learning

Group No.: G__

Topic Selected: AI-Powered Detection and Automated Response to Malicious PowerShell
Attacks Using Machine Learning

Submitted by

Sr. No.	Name of the Student	Roll / Enrolment No.
1	[Student Name 1]	[Roll No.]
2	[Student Name 2]	[Roll No.]
3	[Student Name 3]	[Roll No.]
4	[Student Name 4]	[Roll No.]

Under the guidance of

[Name of the Faculty / Course Coordinator]

[Department Name]
[Name of the Institute]
October 2026

Abstract

PowerShell is installed by default on every supported version of Windows, and the capabilities that make it valuable to system administrators, namely direct access to the .NET runtime and Windows APIs, in-memory execution and remote management, also make it one of the most frequently abused tools in intrusions that “live off the land”. Signature-based defences detect such scripts poorly, since a payload can be renamed, re-encoded or fragmented without any change in behaviour. This report presents the design, implementation and evaluation of a machine-learning pipeline that classifies PowerShell script blocks as malicious or benign and uses the resulting risk score to drive a tiered, automated response. The public MPSD corpus (4,202 malicious and 4,316 benign scripts) was first subjected to a data-quality audit, which showed that 55.2% of the malicious files were exact duplicates, that 12 files carried both labels, and that byte-order marks occurred only in benign files. After cleaning, 1,875 malicious and 4,302 benign scripts remained, and the malicious scripts were found to belong to only 816 structural families. Character 3–5-gram TF-IDF features were combined with 33 interpretable statistical and behavioural indicators aligned with MITRE ATT&CK, and logistic regression, random forest, histogram gradient boosting and a soft-voting ensemble were compared under family-aware five-fold cross-validation. The ensemble obtained an F1-score of 0.972 and a ROC-AUC of 0.998, detecting 97.4% of malicious scripts at a false-positive rate of 1%; a naive random split on the uncleaned data reported 98.8% for the same measure. Malicious code embedded in long benign scripts emerged as the principal weakness (46.7% detected), which improved to 96.4% when comparable samples were added to training, at the cost of a modest increase in false positives. In a replay of 1,233 unseen scripts presented as Windows Event ID 4104 records, the response engine contained 91.2% of malicious events, wrongly contained 0.23% of benign events and required a median of 90.7 ms per event. The results indicate that lightweight, interpretable models combined with a proportionate and human-supervised response policy offer a practical defence against PowerShell-based attacks.

Keywords: PowerShell; malware detection; machine learning; living-off-the-land attacks; TF-IDF; random forest; ensemble learning; MITRE ATT&CK; automated incident response; SOAR; Script Block Logging; AMSI.

Contents

Abstract	ii
1 Introduction	1
1.1 Background and Motivation	1
1.2 Relevance in the Current Threat Landscape	1
1.3 Problem Statement	2
1.4 Objectives	2
1.5 Contributions	2
1.6 Organisation of the Report	3
2 Literature Review	3
2.1 PowerShell as an Attack Vector and Defensive Telemetry	3
2.2 Obfuscation and De-obfuscation	3
2.3 Machine-Learning-Based Detection	4
2.4 Methodological Pitfalls in Security Machine Learning	4
2.5 Automated Incident Response	5
2.6 Summary and Research Gap	5
3 Methodology	5
3.1 System Overview	5
3.2 Threat Model and Assumptions	7
3.3 Dataset and Data-Quality Audit	7
3.4 Pre-processing and Normalisation	8
3.5 Feature Engineering	8
3.5.1 Character n-gram TF-IDF	8
3.5.2 Statistical Features	9
3.5.3 Behavioural Indicator Groups Aligned with ATT&CK	9
3.6 Classification Models	9
3.7 Evaluation Protocol	11
3.8 Automated Response Policy	12
3.9 Ethical and Safety Considerations	12
4 Implementation Details	13
4.1 Development Environment	13
4.2 Software Architecture	13
4.3 Hyper-parameters	13
4.4 Response Engine	13
4.5 Deployment on Windows Endpoints	13
4.6 Verification	15

5	Results and Discussion	15
5.1	Effect of the Data-Quality Audit	15
5.2	Detection Performance under Family-Aware Evaluation	16
5.3	Effect of the Evaluation Protocol	17
5.4	Malicious Code Embedded in Benign Scripts	18
5.5	Interpretability	20
5.6	Error Analysis	21
5.7	Automated Response Simulation	21
5.8	Comparison with Published Work	22
5.9	Limitations and Threats to Validity	23
6	Conclusion and Future Work	24
	References	25

List of Figures

1	Architecture of the proposed detection and automated response system	6
2	Effect of the data-quality audit on the MPSD corpus	8
3	ROC curves of all models under family-aware cross-validation	17
4	Confusion matrix of the ensemble (out-of-fold, threshold 0.5); percentages are relative to the actual class	17
5	Detection rate at 1% FPR under the two evaluation protocols	18
6	Detection of malicious code embedded in benign scripts before and after mixed-aware training	19
7	Twelve most important hand-crafted features of the random forest	20
8	Distribution of out-of-fold ensemble scores with the policy thresholds (*isolation also requires a high-impact technique)	22
9	Response tiers assigned to benign and malicious events in the replay experiment	23

List of Tables

1	Summary of representative related work	6
2	Behavioural indicator groups, example patterns and associated ATT&CK techniques	10
3	Response tiers, triggering conditions and automated actions	12
4	Software and hardware environment	13
5	Main modules of the implementation	14
6	Model hyper-parameters	14
7	Out-of-fold detection performance under family-aware five-fold cross-validation (threshold 0.5; malicious is the positive class)	16
8	Naive random split on raw data compared with the family-aware split on cleaned data	18

9	Embedded-code detection and clean-data performance before and after mixed-aware training	19
10	Outcome of replaying an unseen fold through the response engine (dry-run) . .	22
11	Indicative comparison with published PowerShell detectors (datasets and protocols differ, so the figures are not directly comparable)	23

1 Introduction

1.1 Background and Motivation

PowerShell is a command shell and scripting language built on the .NET runtime that has shipped with Windows since Windows 7 and Windows Server 2008 R2. Administrators rely on it for configuration management, Active Directory maintenance, software deployment and cloud automation. The same characteristics that make it useful for administration, in particular unrestricted access to .NET classes and Win32 APIs, the ability to execute code held only in memory and built-in remoting, are precisely those an adversary requires after obtaining an initial foothold. A short PowerShell command can retrieve a second-stage payload over HTTP, load it by reflection and execute it without writing an executable to disk. Such intrusions are commonly described as *fileless* or *living-off-the-land* attacks, because they rely on software that is already present and trusted on the host [1].

The problem is long-standing. As early as 2016, Symantec reported that 95.4% of the PowerShell scripts submitted to the Blue Coat sandbox it analysed were malicious [2]. MITRE ATT&CK catalogues the behaviour as sub-technique T1059.001 [3], and the 2025 Threat Detection Report of Red Canary identified Command and Scripting Interpreter as the most frequently observed ATT&CK technique for the sixth consecutive year [4]. Removing PowerShell altogether is not a realistic option. In June 2022, the US National Security Agency (NSA) and the Cybersecurity and Infrastructure Security Agency (CISA), together with the national cyber security centres of New Zealand and the United Kingdom, advised organisations to retain PowerShell and to configure and monitor it appropriately [5]. The recommended controls, which include Script Block Logging, the Antimalware Scan Interface (AMSI), constrained language mode and transcription, generate a large volume of useful telemetry. They do not, however, decide whether a particular script is hostile, and that decision must be taken quickly and at scale if an attack is to be interrupted before it achieves its objective. Machine learning (ML) is well suited to this decision.

1.2 Relevance in the Current Threat Landscape

Several developments make an ML-based detection and response capability for PowerShell particularly relevant at present. Obfuscation has become inexpensive and automated: freely available frameworks such as Invoke-Obfuscation can transform a single command into a very large number of syntactically different but functionally equivalent variants [6], which defeats exact-match signatures while leaving statistical regularities that learned models can exploit [7]. Security operations centres are also overloaded; endpoint products raise far more alerts than analysts can examine, and alert fatigue is a recognised cause of missed intrusions [8]. A calibrated risk score allows low-risk events to be handled automatically and concentrates analyst attention where it is most needed. Finally, generative AI has begun to influence both offence and defence. Large language models (LLMs) have been evaluated for de-obfuscating

real malware droppers [9] and for explaining why a PowerShell script is malicious [10], but their cost and latency mean that fast classical detectors remain necessary as a first-line filter. Current incident-response guidance further treats detection and response as an integral part of cybersecurity risk management [11], which favours systems that link a detector to a documented and auditable response policy.

1.3 Problem Statement

Given the text of a PowerShell script block captured on a Windows endpoint, the system should (i) estimate the probability that the script is malicious with a false-positive rate that is acceptable in production, and (ii) select a proportionate response automatically, ranging from logging the event to terminating the process and isolating the host, while leaving the most disruptive actions under human control. The solution should operate on static text only, because executing unknown scripts in order to observe their behaviour is slow and hazardous, and its evaluation should reflect performance on scripts that are genuinely new to the model.

1.4 Objectives

The objectives of this case study are:

1. to audit a public labelled PowerShell corpus for duplicates, label conflicts and spurious artefacts before any model is trained;
2. to engineer features that capture both lexical patterns and behavioural intent, and to map the behavioural features to MITRE ATT&CK techniques;
3. to train and compare several classifiers under an evaluation protocol that prevents near-duplicate scripts from appearing in both training and test data;
4. to design a policy-driven response engine that converts a risk score into graded actions and records every decision in an audit log; and
5. to measure detection quality, response outcomes and processing latency, and to analyse the limitations of the approach.

1.5 Contributions

This work contributes, first, a data-quality audit of the MPSD corpus [12] that quantifies exact duplication, label conflicts and a byte-order-mark artefact that a classifier could exploit as a shortcut. Second, it introduces a family-aware evaluation protocol based on structural skeleton hashing, under which the 1,875 unique malicious scripts form 816 families and each family is confined to a single cross-validation fold. Third, it develops a hybrid detector that combines character n-gram TF-IDF features with 33 interpretable statistical and ATT&CK-aligned behavioural indicators, and compares it with single-view baselines. Fourth, it identifies malicious code embedded in benign scripts as the main weakness of models trained on standalone samples and evaluates a leakage-free mitigation. Fifth, it implements a tiered response engine

that consumes Windows Event ID 4104 records, attaches ATT&CK context, plans containment actions and requires analyst approval before host isolation, and evaluates the complete detection-to-response loop on unseen data.

1.6 Organisation of the Report

Section 2 reviews related work on PowerShell attack detection, de-obfuscation, evaluation methodology and automated response. Section 3 describes the methodology, Section 4 the implementation, and Section 5 presents and discusses the experimental results. Section 6 concludes the report and outlines future work.

2 Literature Review

2.1 PowerShell as an Attack Vector and Defensive Telemetry

From version 5.0 onwards, PowerShell includes several features intended for defenders. Script Block Logging records the text of every script block compiled by the PowerShell engine in the Microsoft-Windows-PowerShell/Operational log as Event ID 4104 [13]. Because logging takes place at compilation time, code that is generated at run time, for example a string that is decoded and then passed to `Invoke-Expression`, is also recorded in its decoded form. AMSI allows any registered anti-malware engine to inspect a script buffer immediately before execution and to block it [14]. Both mechanisms are recommended in the joint NSA/CISA guidance [5]. On the analysis side, the ATT&CK knowledge base provides a common vocabulary for adversary tactics and techniques [15], and it is used in this work to describe what a detected script attempts to do. Industry reports have documented the abuse of PowerShell for almost a decade, describing its use for payload delivery, lateral movement and reconnaissance [2] and its central role in fileless attack techniques [1]; more recent reports continue to rank script interpreters among the most observed techniques [4].

2.2 Obfuscation and De-obfuscation

Bohannon’s `Invoke-Obfuscation` framework demonstrated how easily PowerShell can be disguised at the token, string, encoding and launcher levels [6]. In response, Bohannon and Holmes developed `Revoke-Obfuscation`, which extracts approximately 4,000 features from the abstract syntax tree (AST) of a script and applies a linear model to determine whether it is obfuscated. They assembled a corpus of 408,000 scripts, of which about 7,000 were manually reviewed and labelled [7]. Their work established that statistical properties of code are considerably more robust than string signatures.

A second line of research attempts to reverse obfuscation before detection. `PowerDrive` is a multi-stage static and dynamic de-obfuscator that removes obfuscation layers progressively and was used to analyse thousands of real attacks [16]. Li et al. proposed a light-weight de-obfuscation technique based on AST subtrees, combined with semantic-aware attack detec-

tion [17], which Xiong et al. later generalised [18]. PowerDP predicts the obfuscation types applied to a command, recovers the original content through targeted regular-expression replacement and profiles behaviour with multi-label classifiers [19], whereas PowerPeeler performs dynamic de-obfuscation at the instruction level during execution [20]. Patsakis et al. evaluated LLMs on PowerShell droppers from the Emotet campaign and found that GPT-4 recovered the hidden download URLs in 69.56% of cases [9]. Taken together, these studies show that de-obfuscation improves detection but adds processing cost and complexity. In-engine telemetry such as Event ID 4104 already exposes a partially de-obfuscated view at no additional cost, and this is the input assumed in the present work.

2.3 Machine-Learning-Based Detection

Hendler, Kels and Rubin conducted one of the first large-scale studies, training character-level convolutional and recurrent networks and a natural-language-processing (NLP) baseline on 66,388 commands collected by Microsoft [21]. Their best detector was an ensemble of the NLP model and a convolutional network, because each detected commands that the other missed. In subsequent work they used data obtained through AMSI and contextual embeddings pre-trained on a large unlabelled corpus, reaching a true-positive rate of nearly 90% at a false-positive rate below 0.1% [22]. Microsoft described the deployment of such deep models in its Defender products [23].

Other studies have explored alternative representations. Rusak et al. learned embeddings of AST nodes to classify malicious scripts by family [24]. Song et al. compared token-based and AST-based feature sets with both classical and deep models and reported detection rates of about 98% after feature optimisation [25]. Mimura and Tajiri showed that word-embedding representations support purely static detection [26]. Fang et al. combined FastText embeddings with textual, token and AST-node features in a random forest, reporting 98.93% accuracy on original scripts and 97.76% on a harder set in which malicious code is inserted into benign scripts; they also released the MPSD dataset used in this study [12]. Outside PowerShell, Saxe and Berlin showed that character-level models perform well on short security artefacts such as URLs, file paths and registry keys [27]. Broader surveys of ML for malware analysis are provided by Ucci et al. [28] and Gibert et al. [29]. The most recent direction uses LLMs not only to classify scripts but also to localise the responsible code and explain the verdict in natural language [10].

2.4 Methodological Pitfalls in Security Machine Learning

High reported accuracy does not guarantee practical value. Sommer and Paxson argued that intrusion detection differs from conventional ML tasks because errors are costly, labelled data are scarce and the environment changes continuously [30]. Arp et al. catalogued ten common pitfalls, including sampling bias, data snooping, spurious correlations and inappropriate performance measures, and confirmed in a study of 30 papers from leading security venues that these pitfalls are widespread [31]. Pendlebury et al. demonstrated that random train/test splits which

disregard time and realistic class ratios can substantially overstate the performance of malware classifiers [32]. For imbalanced problems, Saito and Rehmsmeier recommend complementing ROC analysis with precision-oriented measures [33]. These findings shaped the evaluation design described in Section 3.7: duplicated and near-duplicated scripts constitute a form of data snooping, and the byte-order-mark artefact is a clear example of a spurious correlation.

2.5 Automated Incident Response

Security orchestration, automation and response (SOAR) platforms connect detection tools to response actions through playbooks. Islam et al. reviewed the academic and grey literature on security orchestration and identified the integration of heterogeneous tools and the automation of repetitive response tasks as its principal aims [34]. NoDoze reduced false alarms by ranking alerts with the aid of provenance graphs [8], and Hassan et al. subsequently showed that linking endpoint alerts into provenance graphs annotated with ATT&CK tactics and techniques reduces false alarms and supports analyst triage [35]. NIST SP 800-61 Rev. 3 organises incident-response recommendations around the six Functions of the NIST Cybersecurity Framework 2.0 (Govern, Identify, Protect, Detect, Respond and Recover), treating response as part of overall risk management [11].

2.6 Summary and Research Gap

Table 1 summarises representative studies. The literature shows that ML detectors can achieve high accuracy on PowerShell, that obfuscation-resistant statistical features are more valuable than exact strings, and that evaluation choices can substantially inflate reported results. Three gaps remain. Few studies audit their datasets for duplicates and artefacts before reporting results; most work ends at a classification score and does not address how that score should drive a safe automated response; and response decisions are seldom linked to interpretable evidence, such as ATT&CK techniques, that an analyst can verify. The present study addresses all three.

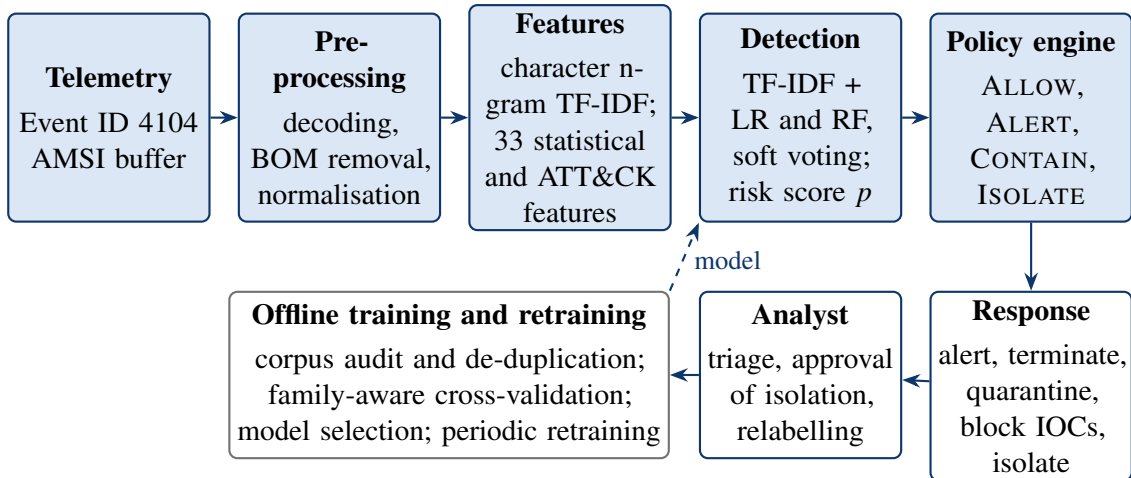
3 Methodology

3.1 System Overview

Figure 1 outlines the proposed system. Script-block text is obtained from endpoint telemetry (Event ID 4104 or an AMSI buffer), cleaned and normalised, transformed into two complementary feature views and scored by a trained model. A policy engine maps the risk score and the behavioural evidence to a response tier, and the associated actions are executed (in this study, simulated) and logged. Analyst decisions are fed back into the labelled data used for periodic retraining. The offline part of the pipeline, comprising the data audit, model training, cross-validation and threshold selection, is described in Sections 3.3–3.7; the online response logic is described in Section 3.8.

Table 1: Summary of representative related work

Study	Approach	Reported outcome	Limitation relevant here
Hendler et al. [21]	Character-level CNN/RNN and NLP ensemble	High recall at very low FPR on 66k commands	Proprietary data; detection only
Hendler et al. [22]	AMSI data, pre-trained contextual embeddings	$\approx 90\%$ TPR at $<0.1\%$ FPR	Requires large unlabelled corpus and GPU training
Bohannon & Holmes [7]	AST features with a linear model	Robust obfuscation detection	Detects obfuscation rather than maliciousness
Li et al. [17]	AST-subtree de-obfuscation and semantic detection	Recovers obfuscated scripts	Additional processing stage; no response
Song et al. [25]	Token/AST feature optimisation	$\approx 98\%$ detection	Random splits; duplicates not analysed
Fang et al. [12]	FastText and hybrid features with RF	98.93% / 97.76% accuracy	Duplicates in data not addressed
Tsai et al. [19]	Obfuscation-type prediction and regex de-obfuscation	98.11% recovery over 15 types	Command level; no response
Wang et al. [10]	LLM classification and explanation	Natural-language explanations	Cost and latency of LLM inference

**Figure 1:** Architecture of the proposed detection and automated response system

3.2 Threat Model and Assumptions

The adversary is assumed to have already obtained code execution on a Windows endpoint, for example through a phishing attachment or stolen credentials, and to use PowerShell to download tools, inject code, establish persistence, access credentials or open a command channel. The adversary may obfuscate scripts, rename variables and embed malicious code within otherwise legitimate scripts. Script Block Logging or AMSI is assumed to be enabled, with its output reaching the analysis service; attempts to disable logging or security tooling are treated as suspicious in their own right through the defence-evasion indicators of Section 3.5. Attacks that never invoke PowerShell are out of scope, as are white-box adversarial attacks in which the adversary can query or inspect the model in order to craft evasive inputs [36]; the latter are discussed as a limitation in Section 5.9.

3.3 Dataset and Data-Quality Audit

The study uses the public MPSD corpus released by Fang et al. [12]. It contains 4,202 malicious scripts (`malicious_pure`), 4,316 benign scripts (`powershell_benign_dataset`) and 4,202 “mixed” malicious scripts, each created by inserting a malicious script into a benign one. The malicious scripts include download cradles, shellcode loaders, reverse shells, credential-theft tools and persistence scripts, while the benign scripts are administrative scripts and modules collected from public repositories.

Following the recommendations of Arp et al. [31], the corpus was audited before training. Each script was hashed with SHA-256 after whitespace normalisation and lower-casing. Only 1,875 of the 4,202 malicious files were unique, so that 55.2% were exact copies of another file; duplicates were removed and each distinct script was retained once. Twelve files, sharing six distinct contents, appeared under both the malicious and the benign label; as their true label cannot be established, all of them were discarded. Finally, none of the malicious files began with a UTF-8 byte-order mark (BOM), whereas approximately 35% of the benign files did. A classifier could therefore learn the rule “a BOM indicates a benign script”, which an attacker could exploit by prepending three bytes; the BOM is consequently removed from every file before feature extraction.

Exact de-duplication does not remove near-duplicates. Many malicious scripts are produced by the same generator and differ only in variable names, numeric constants such as port numbers or shellcode bytes, or whitespace. To capture this, each script d is mapped to a structural skeleton

$$\kappa(d) = \text{SHA256}\left(\text{strip}_{\text{ws}}\left(\text{abs}_{\text{num}}\left(\text{abs}_{\text{var}}\left(\text{lower}(d)\right)\right)\right)\right), \quad (1)$$

in which abs_{var} replaces every variable name by a placeholder, abs_{num} replaces decimal and hexadecimal literals by zero and strip_{ws} removes all whitespace. Scripts with the same skeleton form a *family*. Under this definition the 1,875 unique malicious scripts belong to only 816 families, whereas the 4,302 benign scripts form 4,292 families. Figure 2 shows the effect of each step.

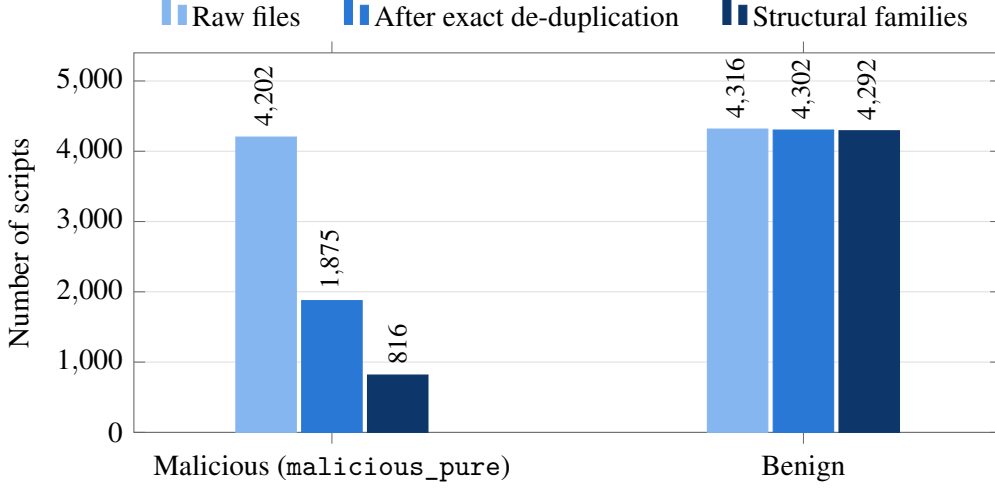


Figure 2: Effect of the data-quality audit on the MPSD corpus

3.4 Pre-processing and Normalisation

Each script is decoded as UTF-8, or as UTF-16 when a UTF-16 BOM is present; any BOM is removed and the text is truncated to 200,000 characters in order to bound processing time for a small number of very large files. For the lexical view the text is further normalised in two ways that preserve its meaning. It is converted to lower case, because PowerShell keywords, cmdlet names and variable names are case-insensitive, and backtick characters are removed, because PowerShell ignores the backtick escape within identifiers, so that a command name interrupted by backticks reduces to its canonical form. Runs of spaces and tabs are collapsed. These inexpensive steps reverse two of the most common token-level obfuscations without executing the script. The statistical features of Section 3.5 are computed on the raw text, since properties such as the proportion of upper-case characters and the density of backticks are themselves informative.

3.5 Feature Engineering

Two complementary views are constructed for every script. The lexical view describes how the code is written, while the statistical and behavioural view describes what the code does and its overall form. The use of complementary views follows the observation of Hendler et al. that models built on different representations tend to make different errors [21].

3.5.1 Character n -gram TF-IDF

The normalised text is decomposed into overlapping character sequences of length three to five. Character n -grams tolerate renamed variables and partially split strings and require no PowerShell parser. The weight of n -gram t in script d uses sub-linear term frequency and smoothed inverse document frequency [37]:

$$w_{t,d} = (1 + \ln \text{tf}_{t,d}) \left(\ln \frac{1 + N}{1 + \text{df}_t} + 1 \right), \quad \hat{\mathbf{x}}_d = \frac{\mathbf{x}_d}{\|\mathbf{x}_d\|_2}, \quad (2)$$

where $\text{tf}_{t,d}$ is the number of occurrences of t in d , N is the number of training scripts, df_t is the number of training scripts that contain t , and $\mathbf{x}_d$ is the vector of weights $w_{t,d}$. Only n-grams that occur in at least three training scripts are retained, up to a vocabulary of 50,000. The vectoriser is fitted within each cross-validation fold on training data only, so that document frequencies never include test scripts.

3.5.2 Statistical Features

Twenty-four numerical features describe the overall form of a script: its length and number of lines (log-scaled); the mean and maximum line length; the proportions of alphabetic, digit, upper-case, whitespace and special characters; the densities of backticks, plus signs (used for string concatenation), quotation marks and braces; the length of the longest token; the number of Base64-like blobs (runs of 80 or more Base64 characters); the numbers of URLs and IPv4 addresses; the density and number of distinct variables; the proportion of comment lines; the number of function definitions; the number of comment-based help keywords; and the presence of a `param()` block. The character-level Shannon entropy [38],

$$H(d) = - \sum_{c \in \Sigma_d} p_c \log_2 p_c, \quad (3)$$

where p_c is the relative frequency of character c in d , is included because encoded or compressed payloads have higher entropy than hand-written code. The documentation-related features reflect the observation that administrative scripts are usually written to be maintained by others and therefore contain help blocks, parameters and functions, whereas attack scripts tend to be short, dense and undocumented.

3.5.3 Behavioural Indicator Groups Aligned with ATT&CK

Nine further features count matches of groups of regular expressions that describe common adversary behaviours (Table 2); each count is log-scaled. Grouping patterns by behaviour instead of using each keyword as a separate feature keeps the feature space small and stable and, more importantly, provides the response engine with human-readable evidence. When a script is contained, the alert can state, for example, that it combined a download cradle (T1105) with dynamic execution (T1059.001). In total, the statistical and behavioural view comprises 33 features.

3.6 Classification Models

Four single-view models and one ensemble were compared, all implemented with scikit-learn [39]. After cleaning, the ratio of malicious to benign scripts is approximately 1:2.3, and every model therefore uses class weights $\alpha_y = n/(2n_y)$, where n_y is the number of training scripts of class y .

Table 2: Behavioural indicator groups, example patterns and associated ATT&CK techniques

Indicator group	Example patterns (normalised text)	ATT&CK technique
Dynamic execution	invoke-expression, iex, invoke-command	T1059.001 PowerShell
Download cradle	downloadstring, downloadfile, net.webclient, invoke-webrequest	T1105 Ingress Tool Transfer
Encoding / compression	frombase64string, -encodedcommand, gzipstream	T1027 Obfuscated Files or Information
Win32 API / injection	virtualalloc, createthread, writeprocessmemory, dllimport	T1055 Process Injection
Defence evasion	-windowstyle hidden, -nopprofile, bypass, set-mpreference	T1562.001 Disable or Modify Tools
Persistence	schtasks, register-scheduledtask, new-service	T1053.005 Scheduled Task
Credential access	mimikatz, sekurlsa, lsass	T1003.001 LSASS Memory
Raw sockets	net.sockets, tcpclient, tcplistener	T1095 Non-Application Layer Protocol
Reflection	reflection.assembly, add-type	T1620 Reflective Code Loading

Logistic regression. For the high-dimensional, sparse TF-IDF view, logistic regression (LR) is a natural choice: it trains quickly, is resistant to over-fitting under L_2 regularisation and its coefficients identify the n-grams that drive a decision. With labels $\tilde{y}_i \in \{-1, +1\}$, the model is obtained by solving

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_{i=1}^n \alpha_{y_i} \log \left(1 + e^{-\tilde{y}_i (\mathbf{w}^\top \mathbf{x}_i + b)} \right), \quad P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b), \quad (4)$$

where σ is the logistic function and $C = 10$. The same model, applied to the 33 standardised hand-crafted features with $C = 1$, serves as an interpretable baseline.

Random forest. A random forest (RF) of $T = 300$ decision trees, each grown on a bootstrap sample with random feature subsets and split by Gini impurity $G = 1 - \sum_k p_k^2$ [40], is trained on the hand-crafted features. Its probability estimate is the average of the tree estimates, $\hat{p}_{\text{RF}}(\mathbf{x}) = \frac{1}{T} \sum_{t=1}^T \hat{p}_t(\mathbf{x})$. Random forests capture non-linear interactions between features, require no scaling and are robust to outliers such as extremely long scripts.

Histogram gradient boosting. A boosted tree ensemble (HGB) that bins features into histograms, in the manner of LightGBM [41], is included as a strong conventional learner for tabular data.

Soft-voting ensemble. The final detector averages the malicious-class probabilities of the lexical and behavioural models:

$$p_{\text{ens}}(\mathbf{x}) = \frac{1}{2} p_{\text{LR}}^{\text{TF-IDF}}(\mathbf{x}) + \frac{1}{2} \hat{p}_{\text{RF}}^{\text{hand}}(\mathbf{x}). \quad (5)$$

Because the two members rely on different evidence, surface character patterns on the one hand and overall form and behaviour on the other, their errors are only weakly correlated. Equal weights were fixed in advance rather than tuned on the evaluation folds, so as to avoid optimistic bias. Deep neural networks were not used: on a corpus of roughly six thousand unique scripts, published results [12, 25] indicate that well-engineered classical models are competitive, and they are cheaper to train, faster at inference and easier to explain to an analyst.

3.7 Evaluation Protocol

All models are evaluated by five-fold cross-validation [42] under two protocols. The *naive* protocol, used only for comparison, applies ordinary stratified five-fold cross-validation to the raw corpus including duplicates; it mirrors a common practice in the literature. The *family-aware* protocol, which provides the main results, applies stratified group five-fold cross-validation to the de-duplicated corpus, using the skeleton family of Equation (1) as the group. All members of a family are thus assigned to the same fold, and no model is tested on a script whose near-copy it has seen during training.

Predictions on the held-out folds are pooled, so that every script is scored exactly once by a model that did not see it during training. Treating the malicious class as positive, the following measures are reported:

$$\begin{aligned} \text{Precision} &= \frac{TP}{TP + FP}, & \text{Recall} = \text{TPR} &= \frac{TP}{TP + FN}, \\ F_1 &= \frac{2 \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, & \text{FPR} &= \frac{FP}{FP + TN}, \end{aligned} \quad (6)$$

together with accuracy, the area under the ROC curve (ROC-AUC) and the true-positive rate at a fixed false-positive rate of 1% (TPR@1%FPR), obtained by linear interpolation of the ROC curve. The last measure is the most relevant operationally: in a large organisation even a 1% false-positive rate on benign scripts can produce many alerts per day, so the detection rate at a low and fixed FPR is a more meaningful summary than accuracy [31, 33].

A third experiment uses the mixed subset of MPSD, in which malicious code is hidden inside a benign script. To avoid leakage, each mixed sample is scored only by the model of the fold that did not contain its malicious parent script. The benign host may have been seen during training with a benign label, which makes the test deliberately demanding. Mixed samples whose malicious parent could not be identified from the file name were excluded.

3.8 Automated Response Policy

Detection alone does not stop an attack. The response engine maps the ensemble score p and the set $\mathcal{G}$ of behavioural indicator groups that fired to one of four tiers:

$$\text{tier}(p, \mathcal{G}) = \begin{cases} \text{ISOLATE} & \text{if } p \geq 0.95 \text{ and } \mathcal{G} \cap \mathcal{H} \neq \emptyset, \\ \text{CONTAIN} & \text{if } p \geq 0.70 \text{ (and not ISOLATE),} \\ \text{ALERT} & \text{if } 0.30 \leq p < 0.70, \\ \text{ALLOW} & \text{if } p < 0.30, \end{cases} \quad (7)$$

where $\mathcal{H}$ contains the high-impact groups (Win32 API/injection, credential access and reflection). The tiers follow the principle of proportionality: inexpensive and reversible actions are automated freely, disruptive actions require stronger evidence, and network isolation of a host additionally requires analyst approval. Table 3 lists the actions associated with each tier. The thresholds were fixed *a priori* from the operational meaning of each tier rather than tuned on test data; Section 5.7 reports how the out-of-fold scores are distributed relative to them.

Table 3: Response tiers, triggering conditions and automated actions

Tier	Condition	Automated actions	Human role
ALLOW	$p < 0.30$	Record verdict in SIEM	None
ALERT	$0.30 \leq p < 0.70$	Record verdict; open a SOC ticket with score, ATT&CK techniques and defanged IOCs	Analyst triage
CONTAIN	$p \geq 0.70$	As above, plus terminate the PowerShell process, quarantine the script file, block its SHA-256 hash and block extracted URLs and IP addresses	Review; may roll back
ISOLATE	$p \geq 0.95$ and a high-impact group present	As above, plus network isolation of the host	Approval required

Every decision, comprising the score, tier, techniques, indicators of compromise (IOCs) and actions, is written as one JSON record to an append-only audit log. IOCs are defanged (for instance, `http` becomes `hxxp` and dots become `[.]`) so that log entries cannot be followed accidentally. By default the engine operates in a dry-run mode in which every action is marked as simulated, which reflects the way SOAR playbooks are normally validated before being permitted to act [34].

3.9 Ethical and Safety Considerations

All malicious samples were handled purely as text and were never executed. The dataset is obtained from its public source at run time and is not redistributed with the project code. IOCs

are defanged in all outputs and are omitted from this report. The response engine cannot perform a real action unless dry-run mode is explicitly disabled, and host isolation always requires human approval, consistent with the principle that automated defences should fail safely and remain accountable to people [11].

4 Implementation Details

4.1 Development Environment

The system was implemented in Python. Table 4 lists the software and hardware used for all experiments.

Table 4: Software and hardware environment

Component	Version / specification
Programming language	Python 3.13
Machine-learning library	scikit-learn 1.9 [39]
Data handling	pandas 3.0, NumPy 2.5
Plotting	Matplotlib 3.11
Testing	pytest (seven unit tests)
Hardware	4 virtual CPU cores, 15 GB RAM, no GPU
Dataset	MPSD, public repository <code>das-lab/mpsd</code> [12]

4.2 Software Architecture

The code is organised as a small Python package and a set of experiment scripts (Table 5). Every model is a scikit-learn Pipeline that accepts raw script text, so the same code path is used in cross-validation, in the response simulation and in deployment. Random seeds are fixed, and all reported figures can be regenerated by fetching the dataset and running the experiment scripts.

4.3 Hyper-parameters

Table 6 lists the hyper-parameters. They were set from common defaults followed by a brief sanity check rather than an exhaustive search on the evaluation folds; a nested search might yield small gains but would increase the risk of over-fitting the limited number of malicious families.

4.4 Response Engine

Algorithm 1 summarises the processing of a single Event ID 4104 record.

4.5 Deployment on Windows Endpoints

In a production deployment, Script Block Logging is enabled through Group Policy (Administrative Templates, Windows Components, Windows PowerShell, *Turn on PowerShell Script Block Logging*) [13]. Event ID 4104 records are forwarded to a central collector by Windows

Table 5: Main modules of the implementation

Directory	File	Responsibility
psdetect/	data.py	Reading scripts, BOM removal, de-duplication, label conflicts, skeleton families
	features.py	Normalisation, statistical features, ATT&CK-aligned indicators
	models.py	Model pipelines and the soft-voting ensemble
	response.py	Scoring, tiers, ATT&CK mapping, IOC defanging, actions, audit log
experiments/	run_experiments.py	Naive and family-aware cross-validation, embedded-code test
	mixed_training.py	Leakage-free mixed-aware training
	simulate_response.py	Replay of a held-out fold as Event ID 4104 records
tests/	test_pipeline.py	Unit tests for features, hashing, tiers and defanging

Table 6: Model hyper-parameters

Model	Settings
TF-IDF + LR	character 3–5-grams, min_df=3, 50,000 features, sub-linear tf; $C = 10$, L_2 penalty, balanced class weights
Hand-crafted + LR	standardised features; $C = 1$, balanced class weights
Hand-crafted + RF	300 trees, Gini impurity, unlimited depth, balanced class weights
Hand-crafted + HGB	300 boosting iterations, learning rate 0.08, balanced class weights
Ensemble	equal-weight average of TF-IDF + LR and hand-crafted + RF (Eq. (5))

Algorithm 1 Processing of one script-block event by the response engine

Input: event e with fields *ScriptBlockText*, *ScriptBlockId*, *Computer*, *ProcessId*, *Path*; trained model M ; policy thresholds

Output: decision record appended to the audit log

```
1:  $t_0 \leftarrow$  current time
2:  $p \leftarrow M.\text{predict\_proba}(e.\text{ScriptBlockText})$ 
3:  $\mathcal{G} \leftarrow \{g : \text{indicator group } g \text{ matches the normalised text}\}$ 
4:  $\tau \leftarrow \text{tier}(p, \mathcal{G})$  ▷ Equation (7)
5:  $\mathcal{T} \leftarrow \{\text{ATTACK}(g) : g \in \mathcal{G}\}$ 
6: if  $\tau \geq \text{ALERT}$  then
7:   extract URLs and IP addresses from the text and defang them
8: end if
9:  $A \leftarrow$  actions prescribed for  $\tau$  (Table 3); mark each as simulated if dry-run
10: if  $\tau = \text{ISOLATE}$  then
11:   flag the decision as requiring analyst approval
12: end if
13: write  $(e.\text{ScriptBlockId}, p, \tau, \mathcal{T}, A, \text{time} - t_0)$  to the audit log
```

Event Forwarding or an endpoint agent. Windows splits large scripts across several 4104 events that share a *ScriptBlockId* and carry *MessageNumber* and *MessageTotal* fields, so the collector reassembles them before scoring. The trained pipeline is serialised and served behind an internal interface, and the action stubs of the response engine are connected to the organisation’s endpoint detection and response (EDR) platform, firewall and ticketing system. Because AMSI exposes the same script content immediately before execution [14], the model could also be invoked from an AMSI provider in order to block high-confidence scripts before they run rather than respond afterwards.

4.6 Verification

Seven unit tests verify that normalisation folds case and backticks; that the feature vector has the expected dimension and finite values; that the indicator groups fire on a typical download cradle but not on a documented administrative function; that two scripts differing only in variable names share a skeleton; that the policy assigns the correct tier for given scores; that isolation is selected only in the presence of a high-impact technique and is flagged for approval; and that IOC defanging behaves as specified. All tests pass.

5 Results and Discussion

5.1 Effect of the Data-Quality Audit

The audit altered the experimental setting before any model was trained. Of the 8,518 raw files, 2,329 exact duplicates and 12 conflicting files were removed, leaving 1,875 malicious and 4,302 benign scripts. The class balance therefore changed from approximately 1:1 in the raw corpus to approximately 1:2.3. More importantly, the 1,875 unique malicious scripts belong to only 816

structural families, so that each malicious template appears on average 2.3 times with cosmetic differences. A split that distributes such families across training and test folds allows a model to succeed by recognising near-copies rather than by learning what makes a script malicious. Section 5.3 quantifies this effect.

5.2 Detection Performance under Family-Aware Evaluation

Table 7 reports the out-of-fold results under the family-aware protocol. All models perform well, which confirms that malicious and benign PowerShell differ substantially in both wording and behaviour, but the differences between the models are informative.

Table 7: Out-of-fold detection performance under family-aware five-fold cross-validation (threshold 0.5; malicious is the positive class)

Model	Acc.	Prec.	Recall	F1	AUC	TPR@1%	FP	FN
Hand-crafted + LR	0.974	0.953	0.960	0.957	0.988	0.941	88	75
Hand-crafted + RF	0.980	0.982	0.952	0.967	0.995	0.957	33	90
Hand-crafted + HGB	0.979	0.984	0.948	0.966	0.996	0.962	29	98
Char TF-IDF + LR	0.985	0.980	0.969	0.974	0.999	0.973	37	59
Ensemble (TF-IDF LR + RF)	0.984	0.984	0.962	0.972	0.998	0.974	30	72

Lexical features constitute the strongest single view: TF-IDF + LR achieves the highest F1 (0.974) and ROC-AUC (0.9986). Character n-grams capture fragments of API names, .NET type names and command-line switches even when these are split across strings or surrounded by unfamiliar code. The 33 hand-crafted features are also effective; the tree-based models reach F1-scores of 0.967 (RF) and 0.966 (HGB) and the highest precision of all models. Logistic regression on the same features is clearly weaker (F1 0.957, 88 false positives), which indicates that interactions between features, captured by trees but not by a linear decision boundary, are important.

The ensemble exchanges a small amount of recall for fewer false alarms. At the default threshold it produces 30 false positives compared with 37 for the lexical model, at the cost of 13 additional false negatives, so that the two F1-scores are almost identical (0.972 and 0.974). At the operating point of greatest practical interest, a 1% false-positive rate, their detection rates are virtually the same (97.39% and 97.33%). The ensemble was nevertheless retained as the detector behind the response engine for two reasons. False positives are what make automated containment costly, since each one may terminate a legitimate administrative task, so the model with the fewest false alarms at a competitive detection rate is preferable. In addition, the random-forest member provides behavioural evidence that can be explained to an analyst more readily than a large set of n-gram weights. Across the five folds, the F1-score of the ensemble ranged from 0.945 to 0.994 (mean 0.972, standard deviation 0.017); the variation arises mainly from which malicious families fall into each test fold.

Figure 3 shows the ROC curves on a logarithmic false-positive axis, which magnifies the low-FPR region that matters in practice. The lexical model and the ensemble dominate the

hand-crafted models below 1% FPR, while hand-crafted LR is weaker across the whole range. Figure 4 gives the confusion matrix of the ensemble: 4,272 of 4,302 benign scripts (99.30%) and 1,803 of 1,875 malicious scripts (96.16%) are classified correctly.

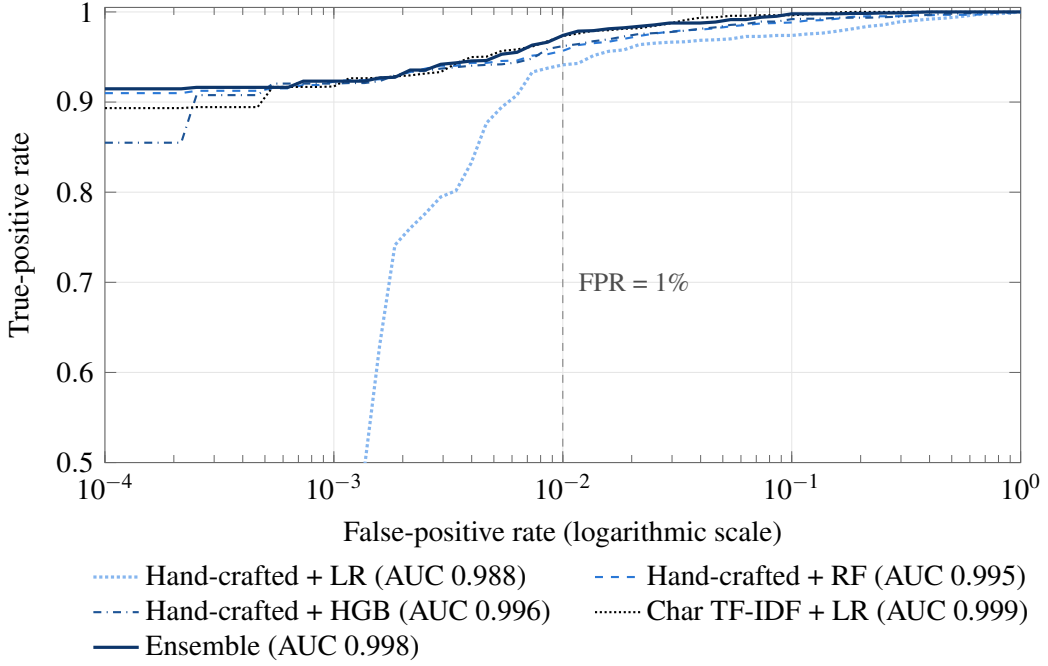


Figure 3: ROC curves of all models under family-aware cross-validation

		Predicted class	
		Benign	Malicious
Actual class	Benign	4,272 (99.30%)	30 (0.70%)
	Malicious	72 (3.84%)	1,803 (96.16%)

Figure 4: Confusion matrix of the ensemble (out-of-fold, threshold 0.5); percentages are relative to the actual class

5.3 Effect of the Evaluation Protocol

Table 8 and Figure 5 compare the two protocols. Under the naive protocol, in which duplicates are retained and folds are random, every model appears better: the F1-score increases by between 0.014 and 0.024, and the TPR@1%FPR of the ensemble rises from 97.39% to 98.83%. The absolute differences appear small, but they conceal a large relative difference in errors. The proportion of malicious scripts missed by the ensemble is 3.84% under the family-aware protocol and 1.69% under the naive one, so a naive evaluation would understate the miss rate on genuinely new attacks by a factor of about 2.3. This is consistent with the warnings of Arp et al. [31] and

Pendlebury et al. [32], and it suggests that results reported on MPSD with random splits should be interpreted with this effect in mind.

Table 8: Naive random split on raw data compared with the family-aware split on cleaned data

Model	F1-score		TPR at 1% FPR	
	Naive	Family-aware	Naive	Family-aware
Hand-crafted + LR	0.981	0.957	0.970	0.941
Hand-crafted + RF	0.985	0.967	0.983	0.957
Hand-crafted + HGB	0.986	0.966	0.983	0.962
Char TF-IDF + LR	0.988	0.974	0.988	0.973
Ensemble	0.988	0.972	0.988	0.974

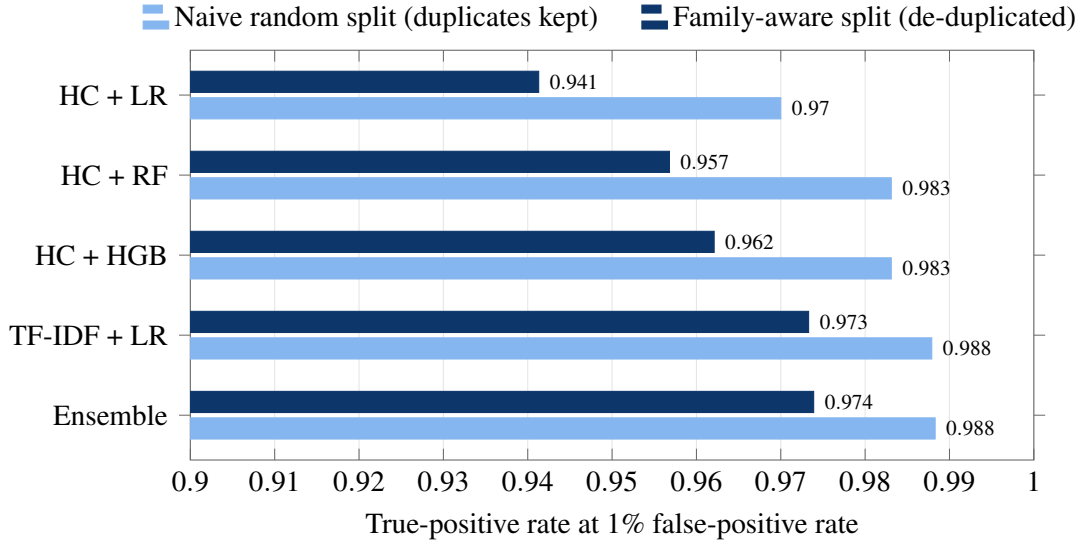


Figure 5: Detection rate at 1% FPR under the two evaluation protocols

5.4 Malicious Code Embedded in Benign Scripts

The mixed subset of MPSD represents a realistic evasion strategy in which a payload is hidden inside a long, legitimate-looking administrative script. In total, 4,064 mixed scripts were evaluated; 138 were excluded because their malicious parent could not be identified. As shown in the first columns of Table 9, every model trained on standalone scripts detected only 41–53% of the embedded payloads at the default threshold, the ensemble detecting 46.7%. This is the most significant weakness revealed by the study. Its cause follows from the interpretability analysis in Section 5.5: the strongest signals are the overall form of a script and, for the lexical model, its length-normalised n-gram profile. When a few lines of malicious code are surrounded by hundreds of lines of neatly formatted administrative code, both views are dominated by the benign host. Hand-crafted LR performs best in this setting (53.4%), because its indicator counts contribute additively rather than being overridden by form-based splits as in the tree models.

To mitigate the weakness, and following Fang et al., who included mixed scripts in training [12], the training folds were augmented with mixed samples. To prevent leakage, a mixed script was added to the training data of fold k only if both its malicious parent and its benign host belonged to the training folds; on average 2,591 scripts were added per fold. Mixed scripts whose malicious parent lay in fold k were again held out for testing. As Table 9 and Figure 6 show, augmentation raised the detection rate of the ensemble on embedded payloads from 46.7% to 96.4%, with 98.3% of these samples reaching at least the ALERT tier; HGB achieved the highest rate (97.4%). The cost is a modest increase in false alarms on ordinary scripts: the false-positive rate of the ensemble rose from 0.70% to 1.05% and its F1-score fell from 0.972 to 0.969. Having seen benign host code inside samples labelled malicious, the model becomes slightly more suspicious of similar benign scripts. Because the tiered policy routes borderline scores to ALERT rather than to automatic containment, this trade-off is considered acceptable, and mixed-aware training is recommended for deployment, subject to re-measuring the false-positive rate on local data.

Table 9: Embedded-code detection and clean-data performance before and after mixed-aware training

Model	Mixed detected		Clean F1		Clean FPR	
	Base	Aug.	Base	Aug.	Base	Aug.
Hand-crafted + LR	53.4%	94.1%	0.957	0.935	2.05%	3.67%
Hand-crafted + RF	47.1%	96.6%	0.967	0.960	0.77%	1.51%
Hand-crafted + HGB	41.1%	97.4%	0.966	0.960	0.67%	1.53%
Char TF-IDF + LR	49.0%	92.1%	0.974	0.969	0.86%	1.12%
Ensemble	46.7%	96.4%	0.972	0.969	0.70%	1.05%

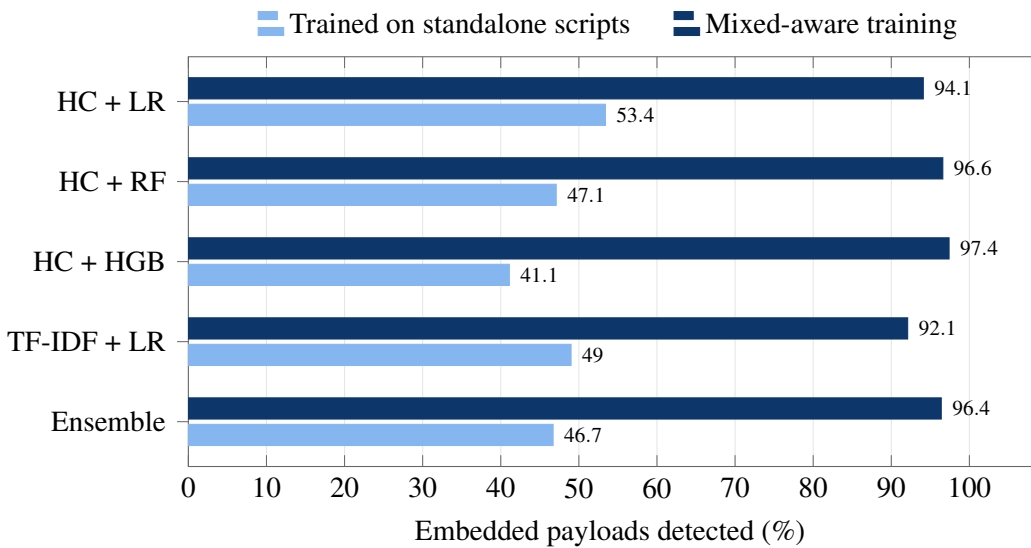


Figure 6: Detection of malicious code embedded in benign scripts before and after mixed-aware training

5.5 Interpretability

Figure 7 lists the most influential hand-crafted features of the random forest. The five most important are the whitespace ratio, the number of lines, the special-character ratio, the mean line length and the digit ratio, all of which describe the form of the code rather than particular commands. The interpretation is straightforward: malicious scripts in the corpus are typically dense one-liners or short blocks with long lines, little indentation, many special characters and long runs of digits, for example shellcode stored as byte arrays, whereas administrative scripts are spread over many short, indented lines. The first behavioural feature, the Win32 API/injection indicator group, ranks eighth. The dominance of form-related features has two sides. Such features are inexpensive and effective, but an attacker can alter them simply by reformatting the code or wrapping it in a long benign script, which is precisely the weakness examined in the previous subsection.

The lexical model provides a complementary and more behavioural explanation. Among its largest positive coefficients are n-grams such as “iex”, “(new-”, “w-ob”, “.exe” and “dow”, which are fragments of the Invoke-Expression alias, of New-Object, of executable file names and of download methods. The strongest benign n-grams, such as “name” and sequences of a brace followed by a variable, reflect well-structured code blocks and named parameters. For an analyst, however, the most useful explanation is the list of ATT&CK techniques attached to every alert by the response engine, because it describes behaviour rather than model internals. Model-agnostic attribution methods such as SHAP [43] could provide per-script explanations in a future version.

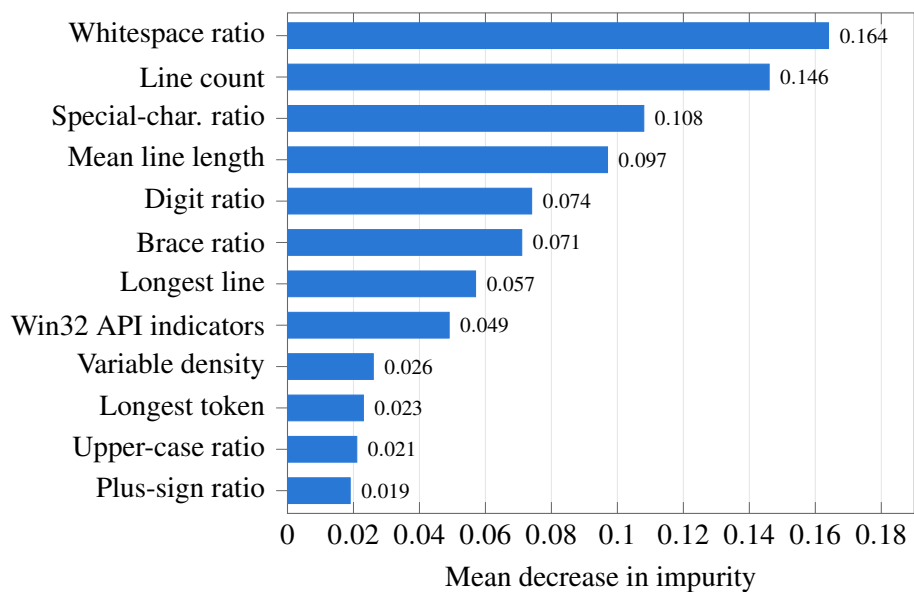


Figure 7: Twelve most important hand-crafted features of the random forest

5.6 Error Analysis

The 30 false positives and 72 false negatives of the ensemble were examined in aggregate, without reproducing their content, and two distinct patterns emerged. The false positives are long, dual-use tools. Their median length is 47,580 characters, more than twenty times that of a typical benign script, and on average 3.13 indicator groups fire per script, compared with 0.32 for correctly classified benign scripts; 70% use reflection and 63% call Win32 APIs. Their function names reveal code-signing helpers, memory and logon-session inspection utilities, DLL-loading wrappers and penetration-testing modules that the dataset labels as benign. Such scripts are genuinely ambiguous, since the same code that an administrator uses for diagnostics is used by attackers for injection.

The false negatives, by contrast, resemble ordinary administration. They contain few behavioural indicators (1.36 groups on average, with 21% triggering none). Typical examples write registry values, create a scheduled task with an innocuous name, collect the user and host name, or build strings from character codes. Whether such actions are malicious depends on context, such as who ran them, from where and what followed, which is not visible in the script text alone. The tiered policy mitigates both types of error: 45.8% of the missed malicious scripts still score at least 0.30 and therefore raise an ALERT instead of passing silently, while 60% of the false positives fall below the CONTAIN threshold and generate only a ticket. Adding context from process ancestry or network telemetry, as in provenance-based triage [8, 35], is the most promising way of reducing both error types further.

5.7 Automated Response Simulation

To evaluate the complete loop from detection to response, the ensemble was trained on four of the five family-aware folds, and the fifth fold was replayed as a stream of synthetic Event ID 4104 records. Each record carried a script-block identifier, host name, user, process identifier and, for 60% of events, a file path. The replay used the base ensemble trained on standalone scripts and therefore measures the response loop on ordinary scripts; the embedded-code case was examined above. Table 10 summarises the outcome.

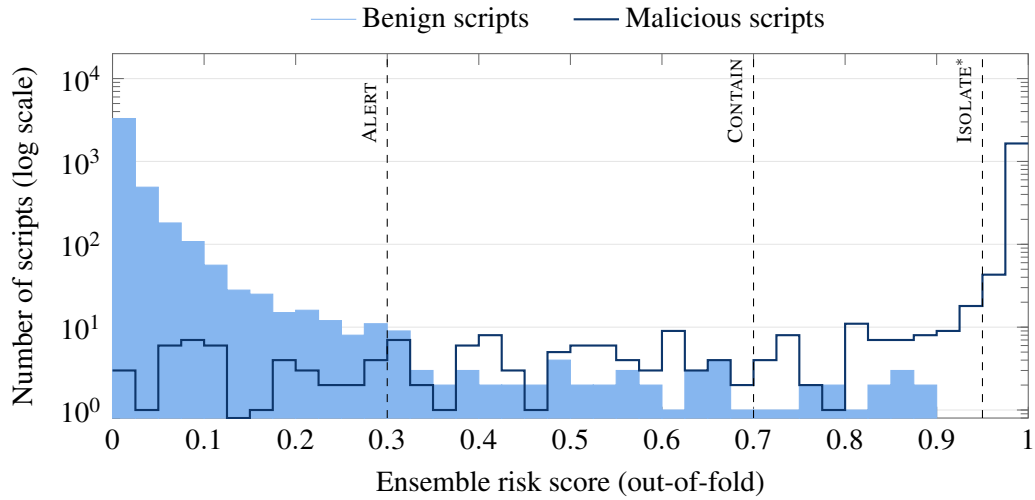
Of the 373 malicious events, 340 (91.2%) were contained automatically, 178 at the CONTAIN tier and 162 at the ISOLATE tier pending analyst approval, and a further 22 raised an ALERT; only 11 (2.9%) passed without any signal reaching an analyst. Of the 860 benign events, 851 (99.0%) were allowed silently, seven produced an alert and two (0.23%) were wrongly contained (Figure 9). Figure 8 explains why the tiers work: the out-of-fold scores of benign scripts are concentrated near zero and those of malicious scripts near one, and the ALERT band between 0.30 and 0.70 captures most genuinely uncertain cases, so automated containment is applied almost exclusively where the model is confident. The large number of isolation requests reflects the prevalence of process-injection loaders in the dataset; in practice, approvals would be grouped by host, because several events usually originate from the same compromised machine.

Every contained event carried ATT&CK context. The techniques attached most frequently

Table 10: Outcome of replaying an unseen fold through the response engine (dry-run)

Measure	Value
Training scripts (folds 1–4) / replayed events (fold 5)	4,944 / 1,233
Replayed malicious / benign events	373 / 860
Malicious: ALLOW / ALERT / CONTAIN / ISOLATE	11 / 22 / 178 / 162
Benign: ALLOW / ALERT / CONTAIN / ISOLATE	851 / 7 / 2 / 0
Malicious events contained (CONTAIN or ISOLATE)	91.15%
Malicious events at least alerted	97.05%
Benign events wrongly contained	0.23%
Benign events raised as ALERT	0.81%
Isolation requests awaiting analyst approval	162
Mean number of actions per contained event	5.41
Latency per event: median / mean / 95th percentile	90.7 / 100.6 / 144.3 ms
Throughput (single process)	9.9 events/s

were T1059.001 (187 events), T1620 (175), T1055 (174) and T1105 (134), which corresponds to the shellcode-loader and download-cradle families that dominate the malicious data. The median latency of 90.7 ms per event, dominated by feature extraction and scoring, allows a single process to handle roughly 35,700 script blocks per hour. Longer scripts take more time, as the 95th percentile of 144.3 ms shows, but remain well below the time an attacker needs to move laterally. Latency was measured in a shared four-core container while other experiments were running and is therefore a conservative estimate.

**Figure 8:** Distribution of out-of-fold ensemble scores with the policy thresholds (*isolation also requires a high-impact technique)

5.8 Comparison with Published Work

Table 11 places the results alongside published work. On the same corpus, the accuracy obtained under the stricter family-aware protocol (98.35%) is close to the 98.93% reported by Fang et al. with random splits, although the present models are simpler and use neither AST parsing

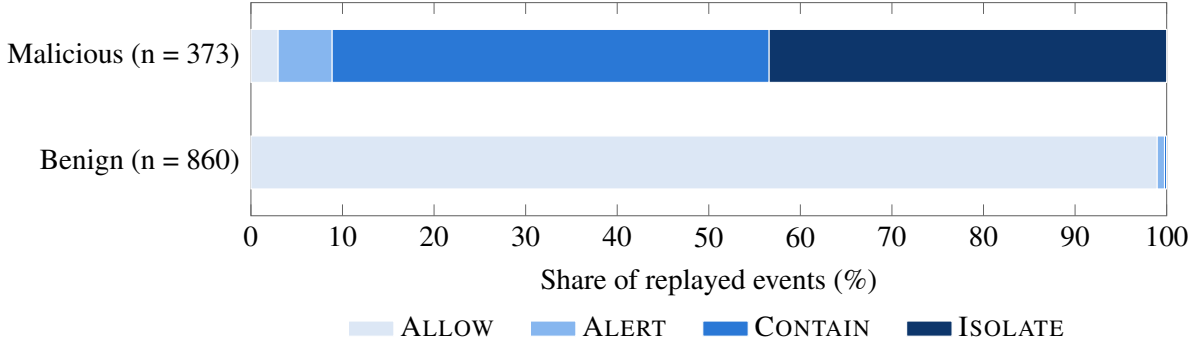


Figure 9: Response tiers assigned to benign and malicious events in the replay experiment

nor embeddings; under a naive split comparable to theirs, the ensemble reaches 98.83%. The comparison with Hendler et al. is only indicative, because their data come from real AMSI telemetry with much greater benign variety, which is a harder and more realistic setting.

Table 11: Indicative comparison with published PowerShell detectors (datasets and protocols differ, so the figures are not directly comparable)

Study	Data and protocol	Reported result
Fang et al. [12]	MPSD; random five-fold CV	Accuracy 98.93% (original), 97.76% (mixed)
Song et al. [25]	Own corpus; token and AST features	Detection rate $\approx 98\%$
Hendler et al. [22]	AMSI data from Microsoft	TPR $\approx 90\%$ at FPR $< 0.1\%$
This work (ensemble)	MPSD, de-duplicated; family-aware five-fold CV	Accuracy 98.35%, F1 0.972, TPR 97.39% at 1% FPR
This work (ensemble)	MPSD raw; random five-fold CV	Accuracy 98.83%, F1 0.988

5.9 Limitations and Threats to Validity

Several limitations should be noted. MPSD is a single public corpus with a limited number of malicious families, and its benign scripts come mainly from public repositories; enterprise environments contain in-house scripts that may look different, so the false-positive rate must be re-measured on local data before deployment. The scripts carry no reliable timestamps, which prevented a temporal split of the kind recommended by Pendlebury et al. [32]; family-aware splitting addresses near-duplicates but not concept drift. The detector sees only script text, whereas context such as the parent process, user, network connections and command-line arguments, which EDR products collect, would help with the ambiguous cases identified in the error analysis. An adversary able to query the model could search for variants that lower the score [36]; using AMSI or Event ID 4104 output, which already removes several obfuscation layers, combining lexical and behavioural evidence and keeping thresholds conservative reduce this risk without eliminating it. Finally, response actions were simulated in dry-run mode on

synthetic event metadata, so the operational cost of terminating processes or isolating hosts in a production network was not measured.

6 Conclusion and Future Work

This case study set out to build and evaluate an AI-based system that detects malicious PowerShell scripts and responds to them automatically, while assessing its performance realistically. A careful audit of the MPSD corpus proved essential: more than half of the malicious files were duplicates, a small number of files carried contradictory labels, byte-order marks offered a spurious shortcut, and the remaining malicious scripts formed only 816 structural families. Under a family-aware evaluation that respects these properties, a lightweight ensemble of a character n-gram logistic-regression model and a random forest on 33 interpretable features achieved an F1-score of 0.972, a ROC-AUC of 0.998 and a true-positive rate of 97.39% at a 1% false-positive rate. A naive random split would have understated the miss rate by a factor of about 2.3, which is a caution for comparisons with published results. The study also revealed a clear weakness: malicious code hidden in long benign scripts evaded models trained on standalone samples about half of the time, because whole-script form features are diluted by the benign host. Adding such examples to training raised the detection of embedded payloads to 96.4%, at the cost of a small increase in false positives on ordinary scripts.

Detection was then linked to action. The tiered response engine consumes the Event ID 4104 records that Windows already produces, attaches ATT&CK techniques and defanged indicators to every decision, and scales its response from logging to process termination, quarantine and network blocking, with host isolation reserved for analyst approval. In a replay of unseen scripts it contained 91.2% of malicious events, wrongly contained 0.23% of benign events and required a median of 90.7 ms per event. These results indicate that interpretable machine learning combined with a proportionate and auditable response policy is a practical means of defending against living-off-the-land PowerShell attacks, consistent with current guidance to retain PowerShell while monitoring it closely [5, 11].

Several extensions would strengthen the system. It should be validated on enterprise telemetry with a temporal split in order to measure concept drift, and retrained periodically using analyst feedback. Scoring individual functions and script blocks in addition to the whole script would prevent a small malicious section from being diluted by a large benign host. AST-based features and pre-trained code embeddings [12, 22] could be compared with the present features under the same family-aware protocol. Integrating the model as an AMSI provider would allow high-confidence scripts to be blocked before execution. Process-tree and network context from EDR provenance graphs could resolve the dual-use cases that account for most errors [35]. An LLM could act as a second-stage assistant that explains ALERT-tier scripts in natural language [10], with the fast classical model retained as the first-line filter. Finally, robustness against adaptive adversaries should be evaluated and improved through adversarially augmented training.

References

- [1] C. Wueest and H. Anand, “Living off the land and fileless attack techniques,” Symantec Internet Security Threat Report (ISTR) Special Report, Symantec Corp., Jul. 2017.
- [2] C. Wueest, “The increased use of PowerShell in attacks,” Symantec Security Response White Paper, Symantec Corp., Oct. 2016.
- [3] The MITRE Corporation, “Command and Scripting Interpreter: PowerShell (T1059.001),” *MITRE ATT&CK Enterprise Matrix*. [Online]. Available: <https://attack.mitre.org/techniques/T1059/001/>
- [4] Red Canary, “2025 Threat Detection Report,” Red Canary Inc., Denver, CO, USA, 2025. [Online]. Available: <https://redcanary.com/threat-detection-report/>
- [5] National Security Agency, Cybersecurity and Infrastructure Security Agency, New Zealand National Cyber Security Centre and UK National Cyber Security Centre, “Keeping PowerShell: Security measures to use and embrace,” Joint Cybersecurity Information Sheet, Jun. 2022.
- [6] D. Bohannon, “Invoke-Obfuscation: PowerShell command and script obfuscator,” presented at DerbyCon 6.0, Louisville, KY, USA, 2016. [Online]. Available: <https://github.com/danielbohannon/Invoke-Obfuscation>
- [7] D. Bohannon and L. Holmes, “Revoke-Obfuscation: PowerShell obfuscation detection using science,” White Paper, Black Hat USA, Las Vegas, NV, USA, Jul. 2017.
- [8] W. U. Hassan, S. Guo, D. Li, Z. Chen, K. Jee, Z. Li and A. Bates, “NoDoze: Combatting threat alert fatigue with automated provenance triage,” in *Proc. Network and Distributed System Security Symp. (NDSS)*, San Diego, CA, USA, 2019.
- [9] C. Patsakis, F. Casino and N. Lykousas, “Assessing LLMs in malicious code deobfuscation of real-world malware campaigns,” *Expert Systems with Applications*, vol. 256, Art. no. 124912, 2024.
- [10] M. Wang, E. Topolovec, B. Arana, N. V. P. S. G. C. Kuladeep, Z. Xi, G. Yan, X. Shu, S. D. Stoller and P. Yang, “Detection and explanation of PowerShell malware with large language models,” Manuscript, Binghamton University, IBM T. J. Watson Research Center and Stony Brook University, 2026. [Online]. Available: <https://www3.cs.stonybrook.edu/~stoller/papers/powershell-malware-2026.pdf>
- [11] A. Nelson, S. Rekhi, M. Souppaya and K. Scarfone, “Incident response recommendations and considerations for cybersecurity risk management: A CSF 2.0 community profile,” NIST Special Publication 800-61 Rev. 3, National Institute of Standards and Technology, Gaithersburg, MD, USA, Apr. 2025.
- [12] Y. Fang, X. Zhou and C. Huang, “Effective method for detecting malicious PowerShell scripts based on hybrid features,” *Neurocomputing*, vol. 448, pp. 30–39, 2021, doi: 10.1016/j.neucom.2021.03.117.

- [13] Microsoft, “about_Locking_Windows,” PowerShell Documentation, Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_locking_windows
- [14] Microsoft, “Antimalware Scan Interface (AMSI),” Win32 Apps Documentation, Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/amsi/antimalware-scan-interface-portal>
- [15] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington and C. B. Thomas, “MITRE ATT&CK: Design and philosophy,” The MITRE Corporation, McLean, VA, USA, Tech. Rep. MTR180360, 2018.
- [16] D. Ugarte, D. Maiorca, F. Cara and G. Giacinto, “PowerDrive: Accurate de-obfuscation and analysis of PowerShell malware,” in *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA 2019)*, Lecture Notes in Computer Science, vol. 11543. Cham, Switzerland: Springer, 2019, pp. 240–259.
- [17] Z. Li, Q. A. Chen, C. Xiong, Y. Chen, T. Zhu and H. Yang, “Effective and light-weight deobfuscation and semantic-aware attack detection for PowerShell scripts,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, London, UK, 2019, pp. 1831–1847.
- [18] C. Xiong, Z. Li, Y. Chen, T. Zhu, J. Wang, H. Yang and W. Ruan, “Generic, efficient, and effective deobfuscation and semantic-aware attack detection for PowerShell scripts,” *Frontiers of Information Technology & Electronic Engineering*, vol. 23, no. 3, pp. 361–381, 2022, doi: 10.1631/FITEE.2000436.
- [19] M.-H. Tsai, C.-C. Lin, Z.-G. He, W.-C. Yang and C.-L. Lei, “PowerDP: De-obfuscating and profiling malicious PowerShell commands with multi-label classifiers,” *IEEE Access*, vol. 11, 2023.
- [20] R. Li, C. Zhang, H. Chai, L. Ying, H. Duan and J. Tao, “PowerPeeler: A precise and general dynamic deobfuscation method for PowerShell scripts,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS ’24)*, Salt Lake City, UT, USA, 2024.
- [21] D. Hendler, S. Kels and A. Rubin, “Detecting malicious PowerShell commands using deep neural networks,” in *Proc. ACM Asia Conf. Computer and Communications Security (ASIACCS)*, Incheon, Republic of Korea, 2018, pp. 187–197.
- [22] D. Hendler, S. Kels and A. Rubin, “AMSI-based detection of malicious PowerShell code using contextual embeddings,” in *Proc. 15th ACM Asia Conf. Computer and Communications Security (ASIA CCS ’20)*, Taipei, Taiwan, 2020, pp. 679–693.
- [23] Microsoft Defender Research Team, “Deep learning rises: New methods for detecting malicious PowerShell,” Microsoft Security Blog, Sep. 3, 2019. [Online]. Available: <https://www.microsoft.com/en-us/security/blog/2019/09/03/deep-learning-rises-new-methods-for-detecting-malicious-powershell/>
- [24] G. Rusak, A. Al-Dujaili and U.-M. O’Reilly, “AST-based deep learning for detecting malicious PowerShell,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, Toronto, ON, Canada, 2018, pp. 2276–2278.

- [25] J. Song, J. Kim, S. Choi, J. Kim and I. Kim, “Evaluations of AI-based malicious PowerShell detection with feature optimizations,” *ETRI Journal*, vol. 43, no. 3, pp. 549–560, 2021.
- [26] M. Mimura and Y. Tajiri, “Static detection of malicious PowerShell based on word embeddings,” *Internet of Things*, vol. 15, Art. no. 100404, 2021.
- [27] J. Saxe and K. Berlin, “eXpose: A character-level convolutional neural network with embeddings for detecting malicious URLs, file paths and registry keys,” arXiv preprint arXiv:1702.08568, 2017.
- [28] D. Ucci, L. Aniello and R. Baldoni, “Survey of machine learning techniques for malware analysis,” *Computers & Security*, vol. 81, pp. 123–147, 2019.
- [29] D. Gibert, C. Mateu and J. Planes, “The rise of machine learning for detection and classification of malware: Research developments, trends and challenges,” *Journal of Network and Computer Applications*, vol. 153, Art. no. 102526, 2020.
- [30] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in *Proc. IEEE Symp. Security and Privacy*, Oakland, CA, USA, 2010, pp. 305–316.
- [31] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro and K. Rieck, “Dos and don’ts of machine learning in computer security,” in *Proc. 31st USENIX Security Symp.*, Boston, MA, USA, 2022, pp. 3971–3988.
- [32] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder and L. Cavallaro, “TESSERACT: Eliminating experimental bias in malware classification across space and time,” in *Proc. 28th USENIX Security Symp.*, Santa Clara, CA, USA, 2019, pp. 729–746.
- [33] T. Saito and M. Rehmsmeier, “The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets,” *PLoS ONE*, vol. 10, no. 3, Art. no. e0118432, 2015.
- [34] C. Islam, M. A. Babar and S. Nepal, “A multi-vocal review of security orchestration,” *ACM Computing Surveys*, vol. 52, no. 2, Art. no. 37, 2019.
- [35] W. U. Hassan, A. Bates and D. Marino, “Tactical provenance analysis for endpoint detection and response systems,” in *Proc. IEEE Symp. Security and Privacy (S&P)*, San Francisco, CA, USA, 2020, pp. 1172–1189.
- [36] I. J. Goodfellow, J. Shlens and C. Szegedy, “Explaining and harnessing adversarial examples,” in *Proc. Int. Conf. Learning Representations (ICLR)*, San Diego, CA, USA, 2015.
- [37] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [38] C. E. Shannon, “A mathematical theory of communication,” *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.

- [39] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot and É. Duchesnay, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [40] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [41] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye and T.-Y. Liu, “LightGBM: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 3146–3154.
- [42] R. Kohavi, “A study of cross-validation and bootstrap for accuracy estimation and model selection,” in *Proc. 14th Int. Joint Conf. Artificial Intelligence (IJCAI)*, Montreal, QC, Canada, 1995, pp. 1137–1143.
- [43] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 4765–4774.